

Self-Extending ADK Agents: Meta Skills That Write SKILL.md Files (Part 3)

来源: <https://lavinigam.com/posts/adk-agent-skills-part3/>

This is Part 3 of a 3-part series on building ADK agents with reusable skills.

- ← *Part 1: What Are Agent Skills?*
- ← *Part 2: File-Based, External Skills, and SkillToolset Internals*
- *5 SKILL.md Design Patterns Every ADK Developer Should Know* →
- *ADK Core Skills* — official skills for building ADK agents

In Parts 1 and 2, we built ADK skills that exist — an inline SEO checklist, file-based and external skills loaded from directories, all wired into a `SkillToolset` with three auto-generated tools. Part 3 closes the loop: the agent writes its own skills.

By the end of this post, you'll know how to:

- Build a **meta skill** that generates new SKILL.md files following the agentskills.io spec
- Understand how **self-extending agents** change the economics of capability building
- Recognize how skills change **agent decision-making**, not just knowledge

[!tip] Tip

- A **meta skill** teaches the agent to generate new SKILL.md files on demand, following the agentskills.io specification
- Self-extending agents can expand their own capabilities without human intervention by writing and loading new skill definitions at runtime
- Skills change the agent's *decision-making*, not just its knowledge — they reshape how the agent approaches problems
- Giorgio Crivellari's ADK governance skill improved code correctness from 29% to 99%, demonstrating the power of well-structured skills

Pattern 4: ADK Meta Skills That Write SKILL.md Files

A **meta skill** is a skill whose purpose is to generate new SKILL.md files — it teaches the agent how to create additional skills on demand, following the [agentskills.io](#) specification. An agent equipped with a meta skill becomes **self-extending**: it can expand its own capabilities without human intervention by writing and loading new skill definitions at runtime.

The first three patterns cover skills that exist — you write them (inline, file-based) or find them (external). Pattern 4 closes the loop: the agent writes new skills itself.

The `skill-creator` is an inline skill whose instructions explain how to write SKILL.md files, and whose `resources` contain the [agentskills.io specification](#) and a working example. This is where the `resources` field from Pattern 1 becomes essential.

```
# agent.py — Pattern 4: Meta Skill

skill_creator = models.Skill(

    frontmatter=models.Frontmatter(

        name="skill-creator",

        description=(

            "Creates new ADK-compatible skill definitions from requirements."

            " Generates complete SKILL.md files following the Agent Skills"

            " specification at agentskills.io."

        ),

    ),

    instructions=(

        "When asked to create a new skill, generate a complete SKILL.md file.\n\n"
```

```

        "Read \`references/skill-spec.md\` for the format
specification.\n"

        "Read \`references/example-skill.md\` for a working
example.\n\n"

        "Follow these rules:\n"

        "1. Name must be kebab-case, max 64 characters\n"

        "2. Description must be under 1024 characters\n"

        "3. Instructions should be clear, step-by-step\n"

        "4. Reference files in references/ for detailed domain
knowledge\n"

        "5. Keep SKILL.md under 500 lines – put details in
references/\n"

        "6. Output the complete file content the user can save
directly\n"

    ),

    resources=models.Resources(

        references={

            # Full content in agent.py – abbreviated here for
readability

            "skill-spec.md": "# Agent Skills Specification
(agentskills.io)...",

            "example-skill.md": "# Example: Code Review Skill...",

        }

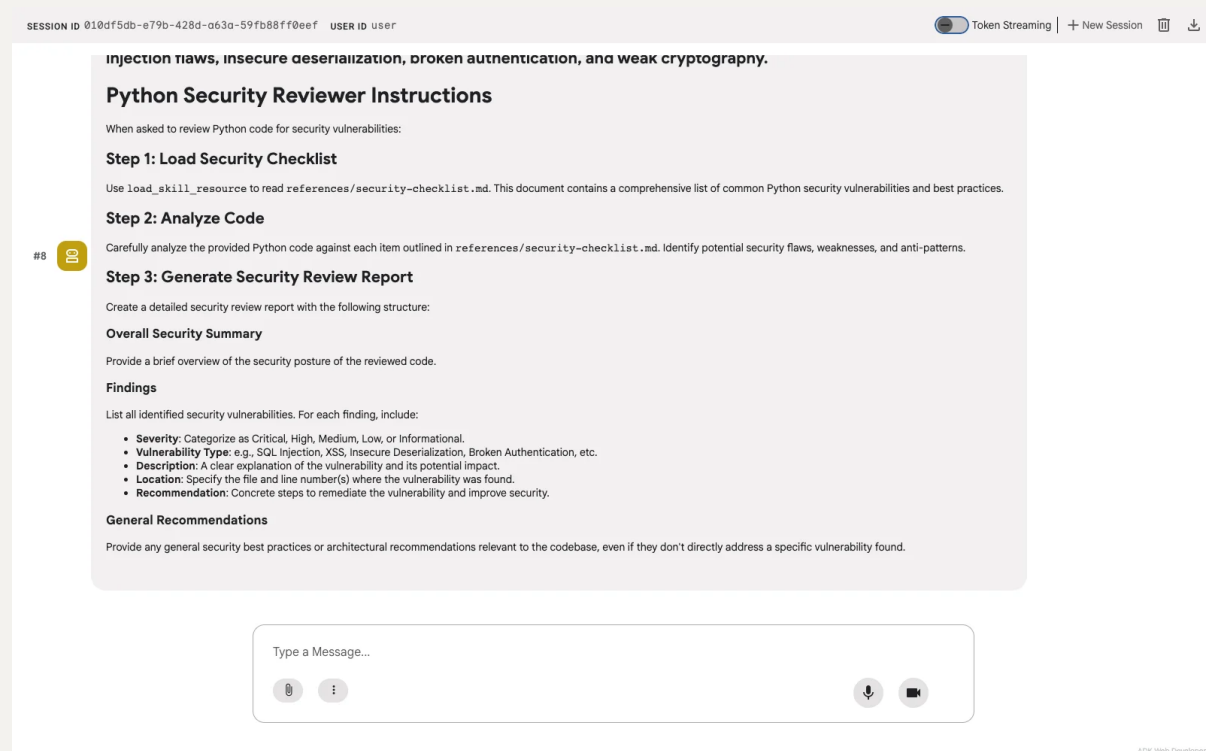
    ),

)

```

The resources embed the agentskills.io spec as `skill-spec.md` and a working code-review skill as `example-skill.md`. When asked to create a new skill, the agent loads the skill-creator, reads both references via `load_skill_resource`, and generates a complete SKILL.md that follows the spec. The pattern is inspired by [obra/superpowers' writing-skills skill](#), which teaches Claude Code how to write new skills.

Lavi Nigam tested this by asking: “I need a new skill for reviewing Python code for security vulnerabilities. Can you create a SKILL.md for it?” The agent loaded the skill-creator, read the spec and example, and generated a complete `python-security-review` skill — valid kebab-case naming, structured instructions covering input validation, authentication, and cryptography, and a severity-based reporting format.



The skill-creator meta skill generates a complete `python-security-review` SKILL.md with valid frontmatter, step-by-step instructions, and a severity-based reporting format.

A skill is just a SKILL.md file. An LLM can generate text. That means an agent with a skill-writing skill becomes self-extending — it can create new domain expertise on demand, then load it via `load_skill_from_dir` in the next session. The generated skills follow the same agentskills.io spec, so they work not just in ADK but in any compatible agent.

What Building ADK Skills Taught Me

Skills change the agent’s decision-making, not just its knowledge. When an agent loads a skill, it doesn’t just gain new information — it gains a new way of approaching problems. The skill reshapes the agent’s reasoning process, not just its reference material.

For a simple SEO review, the agent loaded one skill. For writing, two in parallel. For skill creation, it chained `load_skill` → `load_skill_resource` → `load_skill_resource` → generate. The L1 listing acts as a menu — the agent composes skills based on the task, with no orchestration in the system prompt.

Quality improved more than token usage. With the blog-writer’s style guide as an L3 resource, anti-patterns were eliminated at generation time. Giorgio Crivellari [documented a similar effect](#): his ADK governance skill took code correctness from 29% to 99%.

And the meta skill makes it self-extending — an agent that writes SKILL.md files on demand, following the same agentskills.io spec, working across any compatible agent. Instead of pre-building every capability, build one meta skill and let the agent generate the rest. See for related posts.

[!warning] Warning

When running `adk api_server`, use `adk api_server .` not `adk api_server app/`. ADK looks inside the target directory for sub-applications — if `app/` contains a `skills/` subdirectory, it discovers “skills” as a separate app instead of treating `app/` as your app. Check with `curl localhost:8000/list-apps` if the agent isn’t loading.

Extending ADK Skills: Scripts, Multi-Agent, and Team Libraries

Three directions to explore from here:

- **Script execution** — The ADK source contains a `RunSkillScriptTool` that executes Python and shell scripts from the `scripts/` directory. The docs say “not yet supported,” but the implementation is functional. Skills that can run code, not just provide instructions.
- **Multi-agent + Skills** — A `SequentialAgent` pipeline where a researcher agent loads the content-research-writer skill and feeds into a writer agent with the blog-writer skill. SkillToolset becomes a modular knowledge layer across your entire agent system.
- **Team skill libraries** — Share skills via git repos, version them with tags, load them into any agent that follows the agentskills.io spec. Google does this with [official ADK development skills](#) — a curated library installable via `npx skills add google/adk-docs -y -g` into Gemini CLI, Claude Code, or Cursor. A team-specific library raises the quality floor the way Giorgio’s governance skill did.

Clone the [companion repo](#), install dependencies, set your API key, and run `adk web` .
from the project root to see all four patterns in action. Swap in your own SKILL.md files
under `app/skills/`, or ask the skill-creator to generate one for your domain.



Companion Repository

Clone the repo and run all four skill patterns

→](<https://github.com/lavinigam-gcp/build-with-adk/tree/main/adk-agent-skills-tutorial>) [

ADK Skills Documentation

Official guide for defining and using skills in ADK

→](<https://google.github.io/adk-docs/skills/>) [

Agent Skills Specification

The open standard adopted by 40+ agents

→](<https://agentskills.io/specification>)

Series Summary

Across the [full Agent Engineering series](#), we built a blog-writing agent with four progressively more powerful skill patterns:

1. **Part 1** — The concept: progressive disclosure (L1/L2/L3), project setup, and inline skills defined in Python
2. **Part 2** — The ecosystem: file-based skills with reference docs, external skills from community repos, `SkillToolset` internals, and multi-skill loading
3. **Part 3** — The meta pattern: a skill that writes new skills, self-extending agents, and the quality impact of modular knowledge

The core idea: skills turn a monolithic system prompt into a modular knowledge layer. The agent decides what to load, when to load it, and how to compose multiple skills for complex tasks. That shift — from “everything always present” to “load on demand” — changes not just token efficiency but agent behavior itself.

The ecosystem validates this: Google's [ADK Core Skills](#) use the same SKILL.md format to teach coding agents how to build ADK applications — the specification powering both the tools that write agents and the agents themselves.

Frequently Asked Questions

Can generated skills be loaded in the same session? Yes. After the meta skill writes a new SKILL.md file to disk, you can add its directory to the `SkillToolset` and the agent can use it immediately. In practice, reloading the agent with the new skill directory is the cleanest approach.

How do you version-control generated skills? Treat generated SKILL.md files like any other code artifact — commit them to git. The meta skill writes to a designated output directory, and you review and commit the generated files. This preserves the audit trail and lets you refine generated skills over time.

What prevents the agent from generating harmful or incorrect skills? The meta skill's instructions define the output format and quality constraints. You can add guardrails like requiring specific sections, mandating safety disclaimers, or restricting the domains the agent can generate skills for. The generated SKILL.md files are also human-readable, so you review them before deployment.

Does the meta skill work with models other than Gemini? The meta skill pattern is model-agnostic — it generates text files following the agentskills.io specification. Any LLM that can follow structured output instructions can generate valid SKILL.md files. ADK itself supports multiple model backends.

How do ADK Core Skills relate to the meta skill pattern? ADK Core Skills are [official skills](#) covering ADK development, evaluation, deployment, and observability. The meta skill pattern from this post generates *new* skills on demand. They complement each other: Core Skills provide baseline ADK knowledge, while the meta skill-creator generates domain-specific skills your team needs beyond what Google provides.

References

1. [Skills for ADK Agents](#) — Official ADK documentation for SkillToolset and progressive disclosure
2. [Agent Skills Specification](#) — The open standard defining SKILL.md format, adopted by 40+ agents
3. [What Are Agent Skills?](#) — Conceptual overview of skills as reusable, agent-

agnostic capabilities

4. [ADK Overview \(Vertex AI\)](#) — Google Cloud documentation for Agent Development Kit
5. [skills/models.py](#) — `Skill`, `Frontmatter`, `Resources` class definitions
6. [tools/skill_toolset.py](#) — `SkillToolset` implementation with auto-generated tools
7. [skills/_utils.py](#) — `load_skill_from_dir`, validation, SKILL.md parsing
8. [skills_agent](#) sample — Official ADK sample with inline + file-based skills
9. [awesome-claude-skills](#) — 100+ production skills organized by domain
10. [content-research-writer](#) — The external skill used in this tutorial
11. [writing-skills \(obra/superpowers\)](#) — Meta skill-writing pattern that inspired Pattern 4
12. [adk-skill \(Giorgio Crivellari\)](#) — Open-source ADK governance skill
13. Giorgio Crivellari — [I Built an Agent Skill for Google's ADK](#) — 245% quality improvement case study
14. Ravi Chaganti — [Google ADK Agent Skills](#) — Practical walkthrough
15. Sid Bharath — [The Complete Guide to Google's ADK](#)
16. Spring AI — [Generic Agent Skills](#) — Java ecosystem adopting the spec
17. [@antigravity](#) — [Intro to Agent Skills](#) — The video that sparked this build (85K views)
18. [@liamottley_](#) — [“SaaS is being replaced by SKILL.md files”](#) — 562 reactions
19. [@Pavan_Belagatti](#) — [MCP vs Skills](#) — Clear MCP/Skills differentiation
20. [@dAAAb](#) — [15 Open-Source Agent Skills](#) — 40+ agents supporting agentskills.io
21. [@alexalbert_](#) — [Agent Skills as open standard](#)
22. [ADK Core Skills](#) — official skills for building ADK agents